INTELLIGENT GIS-BASED URBAN SERVICES PLATFORM IN ASTANA

Full Project Documentation

Diploma Project • Software Engineering • SE-2412

UNIVERSITY	Astana IT University
PROJECT TEAM	Айдаров Арлан Адилханулы — Developer Аширбек Рамазан Толегенулы — Developer Булатов Кайрат Булатович — Developer
SUPERVISOR	Algys Saltanat
DOCUMENT	Project scope, requirements, design, prototype status, verification and next steps

This document distinguishes the current prototype from planned implementation and evaluation. It does not present unverified features or results as completed.

Table of Contents

1. Executive Summary
2. Project Background
3. Problem Statement
4. Aim, Research Question and Objectives
5. Stakeholders and User Roles
6. Scope and Boundaries
7. Functional Requirements
8. Non-Functional Requirements
9. Use Cases and Main Workflows
10. System Architecture Overview
11. Technology Choices
12. Data Model and Data Management
13. GIS Functionality
14. AI Module 1: Hazard Report Classification
15. AI Module 2: Lost-and-Found Matching
16. AI Module 3: Parking Occupancy Prediction
17. User Interface and Interaction Design
18. Security, Privacy and Ethics
19. Testing and Verification Plan
20. Evaluation Methodology
21. Project Management and Team Roles
22. Repository Structure and Version Control
23. Build, Run and Development Setup
24. Risks and Mitigation
25. Current Limitations and Known Gaps
26. Work Plan and Next Steps
27. Expected Outcomes and Success Criteria
28. Conclusion
29. Glossary
30. References and Documentation Notes

The sections cover the project lifecycle from problem definition through implementation planning, testing and documentation control.

1. Executive Summary

Purpose

This document consolidates the project concept, proposed system behaviour, current prototype scope, design direction, development workflow and planned evaluation for the Intelligent GIS-Based Urban Services Platform in Astana.

Current stage

The project has an established topic and research direction, initial documentation and a React/TypeScript interface prototype. The prototype demonstrates selected workflows with schematic map elements and sample data. It is not a deployed municipal service.

Intended value

The proposed platform aims to bring selected urban-service scenarios into one map-oriented interface. A shared spatial context may help users explore parking information, hazard reports, lost-and-found cases and related city information more consistently.

Boundary

Production authentication, persistent backend storage, live city data feeds and validated AI models are not claimed as operational features unless confirmed in the repository.

2. Project Background

Urban-service context

Cities generate information through transport and parking operations, public reports, weather alerts, events and maintenance processes. When information is separated across systems, users may need to switch interfaces and manually connect details.

GIS approach

A geographic information system gives records a spatial context. Locations, markers, layers and overlays can help users understand where events occur and how they relate to other services.

Astana context

Astana is the target context for this diploma project. City-specific operational claims should be supported by verified data during implementation and evaluation.

3. Problem Statement

Problem

Urban-service information can be fragmented across separate interfaces and workflows. This may make it harder to review relevant information together, prioritise reports and compare possible responses.

Proposed response

The platform explores a unified interface for selected service scenarios, supported by GIS-oriented presentation and three planned AI-assisted modules.

Scope boundary

The project does not aim to replace every municipal information system. The first version focuses on a limited set of use cases that can be implemented and evaluated within the diploma project.

Assumptions

The team must validate whether the workflows are useful, whether suitable data can be obtained, and whether AI assistance adds measurable value compared with a baseline.

4. Aim, Research Question and Objectives

Aim

To design, implement and evaluate a prototype of an intelligent GIS-based platform for urban-service analysis and decision support in Astana.

Research question

To what extent does an AI-enhanced, GIS-based platform that integrates parking and reservation, hazard reporting, and lost-and-found services improve urban service analysis and decision support compared with a baseline workflow?

Objectives

Define the scope and use cases; document functional and non-functional requirements; prepare system and data designs; implement a demonstrative web interface; investigate suitable approaches for planned AI modules; and define an evaluation process.

Expected contribution

The expected contribution is a documented prototype and an evaluation of selected workflows. Benefits should be judged using evidence from implementation and testing.

5. Stakeholders and User Roles

Residents and visitors

Potential users may need to view public information, check parking indicators, review hazard reports or submit a report. The interface should avoid unnecessary steps and explain the status of submitted information.

Service operators

Operators may need to review reports, identify duplicates, prioritise issues and understand geographic distributions. These roles should be confirmed with stakeholders.

Project team

The team is responsible for implementation, documentation, testing and transparent reporting of limitations. Responsibilities should be tracked through issues and repository changes.

Supervisor

The supervisor provides academic guidance. Any record of feedback should reflect actual comments and decisions.

6. Scope and Boundaries

Initial concept

The project considers parking and reservation, hazard reporting, lost-and-found matching and related urban information presented through a GIS-oriented interface.

Prototype screens

The current prototype is described as including a schematic city map with parking markers and hazard overlays, parking indicators and filters, approved hazard reports, a hazard-report form, weather alerts, event information and a moderation view.

Not confirmed as implemented

A deployed city service, verified live feeds, production accounts, complete municipal integrations and proven city-wide AI predictions are not established as implemented.

Scope control

New features should be accepted only when their value, implementation cost, data requirements and effect on evaluation are understood.

7. Functional Requirements

FR-01 Map overview

The interface should provide a central view that places selected service information in a spatial context. In the current prototype, the map is schematic and uses demonstration elements.

FR-02 Parking

Users should be able to inspect parking indicators and use available filters. Reservation functionality should not be described as operational unless supported by the implementation.

FR-03 Hazard reports

Users should be able to view approved reports and access a report-submission form. Report states should be clearly distinguished where implemented.

FR-04 Moderation

A moderation view is intended to support review before public display. The team should document moderation rules and test cases.

FR-05 City information

Weather alerts and event information may be shown as demonstration content. Live integration is future work unless verified.

FR-06 Lost and found

The planned service should compare lost-item and found-item descriptions and present likely matches. This remains planned until implemented and tested.

8. Non-Functional Requirements

Usability

Core tasks should be understandable without extensive instructions. Labels, status messages and form feedback should be consistent.

Performance

The prototype should remain responsive for the expected demonstration dataset. Performance claims must be based on measured tests with documented conditions.

Reliability

Errors should be handled clearly. The interface must not imply that an action succeeded if it did not.

Maintainability

Code should be organised into understandable components. Documentation and diagrams should be updated when implementation changes.

Security and privacy

Credentials and personal data must not be committed to the repository. Real user data would require access control, retention and privacy rules.

Accessibility

Text contrast, keyboard use, form labels and clear feedback should be reviewed before user testing.

9. Use Cases and Main Workflows

View city information

The user opens the map overview, reviews visible markers and selects an item. The interface should show enough detail without overwhelming the map.

Check parking

The user reviews parking indicators and applies filters. The system must indicate whether availability is live, sample-based or manually entered.

Submit a hazard report

The user opens the form, enters required details and submits it. A complete workflow validates input and communicates the submission status.

Moderate a report

A reviewer inspects a pending report and accepts, rejects or requests clarification under documented rules. A deployed system requires role-based access control.

Find a lost item

The user enters a description and receives candidate matches ranked by relevance. A candidate is a possible match, not proof of ownership.

10. System Architecture Overview

Current direction

The demonstrative web interface is built with React and TypeScript, supporting reusable components and a structured client-side application.

Logical layers

The intended architecture can be described as user interface, application services, data storage/integration, and analytics/AI. This conceptual separation should be aligned with the final architecture diagram.

Interface layer

The interface presents maps, filters, forms, alerts and moderation controls. Components should remain small enough to maintain and test.

Application services

Future services may handle report validation, parking queries, moderation actions and lost-and-found matching. The backend design remains a decision to document.

Data and AI

The data layer should provide consistent records and provenance. AI modules should be separated from interface code so that they can be tested independently.

11. Technology Choices

Frontend

React and TypeScript are used for the current prototype. React supports component-based development; TypeScript can catch type-related errors during development.

Backend candidates

FastAPI and Django REST Framework are candidates for a future Python backend. FastAPI may suit typed APIs and lightweight services; Django REST Framework offers an integrated framework with models, authentication and administration tools.

Decision criteria

Compare framework fit, development speed, validation, authentication, database integration, documentation, testability and team familiarity. Record the final choice and rationale.

Database

Persistent storage will be needed if records must survive refreshes or be shared. Selection should reflect relationships, geospatial needs, deployment and team capacity.

Mapping

The current prototype uses schematic map presentation. A production map solution requires decisions about tiles, geospatial data, licensing, performance and accuracy.

12. Data Model and Data Management

Potential entities

The conceptual model may include User, ParkingLocation, ParkingStatus, Reservation, HazardReport, LostItem, FoundItem, Event and WeatherAlert. The final schema should include only entities needed for implemented scope.

Identifiers and timestamps

Persistent records should use stable identifiers and consistent timestamps, with a defined time-zone policy.

Data provenance

Each dataset should be labelled as real, synthetic, manually entered or externally sourced. Record collection date, permissions, transformations and limitations.

Data quality

Validation should check required fields, valid locations, duplicates and inconsistent statuses. Issues should be reported rather than silently hidden.

Synthetic data

Synthetic data supports repeatable interface testing but cannot establish real-world model performance.

13. GIS Functionality

Spatial context

GIS functionality is intended to help users understand the location and distribution of service records. Marker meaning and status should be clear.

Coordinate validation

Future integration should validate coordinate ranges, coordinate reference systems and location precision.

Layers and filters

Potential layers include parking locations, hazard reports and service areas. Filters should be understandable and provide feedback.

Geospatial analysis

Future analysis may include clustering, distance queries and area summaries. These require sufficiently reliable data.

Limitations

A schematic map can demonstrate interaction concepts but should not be represented as an accurate navigation or operational map.

14. AI Module 1: Hazard Report Classification

Purpose

The planned module would assign incoming hazard reports to predefined categories to help organise records for review.

Input and output

Possible input includes report text and selected metadata. Output may include a predicted category and confidence score. Categories must be defined before evaluation.

Workflow

Prepare a labelled dataset, split training and test data, establish a baseline, train or configure a classifier, measure performance and review errors.

Evaluation

Macro-F1 is a possible metric when categories have unequal frequencies. Per-class precision and recall should also be reviewed.

Risks

Ambiguous text, class imbalance, language variation and sensitive content may reduce performance. Human review should remain available for uncertain cases.

15. AI Module 2: Lost-and-Found Matching

Purpose

The planned module would help users identify potentially related lost and found item records.

Features

Candidate features may include item category, description, approximate time, location and distinctive attributes. Only appropriate, available fields should be used.

Ranking workflow

Retrieve candidates, calculate similarity and return a ranked list. The interface should explain that a suggested match is not proof of ownership.

Evaluation

Top-5 retrieval rate can measure whether a known correct match appears among the first five candidates. The dataset should contain realistic positive and negative examples.

Privacy

Descriptions should avoid unnecessary personal information. Verification steps should reduce false claims and protect owners.

16. AI Module 3: Parking Occupancy Prediction

Purpose

The planned module would estimate parking occupancy or availability for a future time window.

Data requirements

Useful data may include timestamped occupancy observations, location identifiers, capacity and contextual variables. Reliable observations are a key dependency.

Baseline

A simple baseline could use the latest known occupancy or a historical average. A complex model should be used only if it improves on the baseline.

Evaluation

Mean Absolute Error (MAE) is a possible metric. The prediction horizon, unit, test period and treatment of missing data must be specified.

Limitations

Synthetic data can test the pipeline but cannot demonstrate predictive accuracy in Astana.

17. User Interface and Interaction Design

Design principle

The interface should make primary tasks easy to locate and use consistent patterns for markers, status labels, forms and feedback.

Map view

The map should remain the main spatial overview. Marker types need a clear legend; real datasets may require clustering or filtering.

Forms

Forms should identify required fields, validate input and show understandable errors. Users should receive clear feedback after submission.

Moderation view

Moderation controls should distinguish pending, approved and rejected items. A deployed system needs role-based permissions and an audit trail.

Prototype boundary

The prototype demonstrates selected layouts and flows but does not establish that backend logic, persistence or integrations are complete.

18. Security, Privacy and Ethics

Data minimisation

Collect only information necessary for the stated use case. Avoid personal identifiers in demonstration datasets unless essential and protected.

Access control

If accounts or moderation roles are added, permissions should be enforced by the backend rather than hidden interface controls.

Secrets

API keys, passwords, tokens and private configuration must not be committed. Use environment configuration and an appropriate ignore file.

User study

Before recruiting participants, confirm ethics, consent, data handling and retention requirements with the supervisor.

AI transparency

Where AI affects prioritisation or matching, communicate uncertainty and provide a way to correct or challenge errors.

19. Testing and Verification Plan

Unit tests

Test form validation, filtering, status changes and data transformations. Test AI preprocessing and scoring separately.

Integration tests

When APIs and storage exist, test the full path from interface action to service response and persistence.

End-to-end tests

Create repeatable scenarios for checking parking, submitting a report, moderating a report and searching for a lost item.

Usability tests

Give participants defined tasks, observe completion and collect feedback with an agreed method. Do not report results before testing.

Defect tracking

Record reproduction steps, expected and actual behaviour and status. Link fixes to commits or tasks.

20. Evaluation Methodology

Comparison

Define the baseline workflow before testing and apply comparable tasks and conditions.

Technical metrics

Potential measures include macro-F1, top-5 retrieval rate and MAE. Each requires a dataset, calculation method and test split.

User metrics

Potential measures include task-completion rate, task success, time on task and the System Usability Scale. Document recruitment, consent and procedure.

Validity

Threats include small samples, synthetic data, biased labels and limited scenarios. Discuss these in the final report.

Reporting

Report actual values and test conditions. If a module is incomplete or data is insufficient, state that directly.

21. Project Management and Team Roles

Team

SE-2412: Айдаров Арлан Адилханулы — Developer; Аширбек Рамазан Толегенулы — Developer; Булатов Кайрат Булатович — Developer.

Work allocation

Responsibilities should be assigned through issues and reviewed regularly. Possible areas include interface/integration, data/AI experiments, and documentation/testing, with peer review across the team.

Task tracking

Each task should have an outcome, acceptance criteria, owner and status. Large tasks should be divided into reviewable items.

Coordination

Agree on coding conventions, file organisation, branch names, commit messages and review process.

Supervisor review

Meeting notes and feedback records must reflect actual interactions and identify resulting changes.

22. Repository Structure and Version Control

Repository purpose

The repository stores application code, documentation, diagrams and supporting materials needed to review the project state.

Suggested structure

Use src/ for application code, docs/ for documentation, diagrams/ for diagrams and tests/ for tests where appropriate. Actual structure should match the repository.

Branches

Use issue-linked branches such as DIP-13-pr-workflow-access and keep main as the shared integration branch.

Commits

Use concise messages with an issue identifier, for example: DIP-13: Document PR workflow and access.

Pull requests

A PR should explain purpose, changes and verification, identify the related task, and receive review before merging when required.

Tags

A tag such as prob-period-1 should point to a reviewed snapshot and should not be moved casually after submission.

23. Build, Run and Development Setup

Prerequisites

The interface uses a JavaScript/TypeScript toolchain. The exact Node.js version and package manager should be taken from repository configuration.

Install

For an npm-based project, install dependencies from the repository root with npm install. The lockfile helps keep dependency versions consistent.

Run locally

Use the development command defined in package.json, commonly npm run dev for a Vite project. Verify the exact script before treating it as official.

Build verification

Run configured build, lint and test scripts if present. Record the commands actually executed and their outcomes.

Troubleshooting

If the server fails, check terminal output, Node.js version, dependency installation and port configuration. Do not delete configuration files without understanding their role.

24. Risks and Mitigation

Data availability

Real parking or hazard data may be unavailable. Identify sources early, document permissions and use synthetic data only for controlled tests.

Scope growth

Too many functions and AI modules may exceed available time. Prioritise a small end-to-end workflow and implement modules incrementally.

Model performance

AI may not outperform simple baselines. Define baselines early, measure results honestly and retain a non-AI fallback.

Integration

Frontend, backend, GIS and AI may be difficult to integrate. Define API contracts and test integration early.

Evaluation delays

Ethics approval or recruitment may take time. Discuss requirements early and prepare an academically appropriate fallback plan.

25. Current Limitations and Known Gaps

Prototype data

The interface uses schematic or demonstration data and should not be described as displaying verified live city information.

Persistence

Client-side state may reset after refresh. Server-side persistence must be confirmed before claiming that records survive sessions.

Backend and integrations

A production backend, authentication, external municipal APIs and complete GIS services are not established as implemented by the current project description.

AI status

The three AI modules are planned components. Their inclusion in design documents does not mean trained models or validated metrics exist.

Consistency

Diagrams, requirements, setup instructions and report must be checked against the same repository snapshot.

26. Work Plan and Next Steps

Step 1: Requirements

Review use cases with the supervisor, choose mandatory functions and define acceptance criteria.

Step 2: Design

Update system context, use-case, component and data diagrams to match the planned implementation.

Step 3: Prototype

Stabilise the main interface flows and test critical interactions.

Step 4: Data services

Choose backend and storage, define API contracts and implement minimum required data operations.

Step 5: AI modules

Choose datasets and baselines, implement modules incrementally and measure them independently.

Step 6: Evaluation

Run tests, record results and limitations, update documentation and prepare a reviewed submission snapshot.

27. Expected Outcomes and Success Criteria

Prototype

A coherent demonstration of selected urban-service workflows with documented boundaries and repeatable scenarios.

Engineering

A repository that can be reviewed and, where supported by configuration, run locally using documented steps.

Research

A reproducible evaluation plan and actual results for implemented workflows and modules.

Quality

Requirements, diagrams, code and report should describe the same system. Unimplemented features must be marked as planned.

Success criteria

Success should be judged through agreed acceptance criteria, verified functionality and measured results—not simply by the number of screens or planned modules.

28. Conclusion

Summary

The project explores how a shared map-oriented interface and AI-assisted analysis could support selected urban-service workflows in Astana.

Work to date

The project has a defined topic and research direction, initial documentation and a React/TypeScript prototype describing selected map, parking, hazard-report, alert, event and moderation interactions.

Remaining work

Further implementation is needed for data services, GIS integration, AI modules, testing and evaluation.

Final note

The final diploma work should have a realistic scope, transparent limitations, a documented implementation and an evaluation based on measured evidence.

29. Glossary

GIS

Geographic Information System: tools and methods for managing, analysing and presenting information linked to locations.

GeoAI

The use of artificial intelligence methods with geographic or spatial data.

API

Application Programming Interface: a defined way for software components to exchange requests and responses.

Macro-F1

The unweighted average of F1 scores across classes, useful when each class should contribute equally to the summary metric.

MAE

Mean Absolute Error: the average absolute difference between predicted and observed numerical values.

Top-5 retrieval rate

The proportion of queries for which the correct match appears among the five highest-ranked candidates.

Prototype

An early implementation used to demonstrate selected concepts; it is not necessarily a complete or deployable system.

Baseline

A reference method or workflow used for comparison.

Pull request

A request to review and merge changes from one Git branch into another.

Annotated tag

A Git tag containing metadata such as a message and tagger information, often used to identify a submission snapshot.

30. References and Documentation Notes

Project artefacts

The README, source code, configuration, requirements, diagrams, task board and test records should be used as evidence of what has actually been implemented.

Research background

The literature review should cite verified sources on smart-city data platforms, GIS and GeoAI, digital twins, urban-service systems and evaluation of AI-supported decision tools.

Version consistency

The submitted PDF should match the required repository tag. Verify that the tag exists, files are present and the report matches that snapshot.

Accuracy

Claims about completed features, meeting records, test outcomes, data access or supervisor feedback should be confirmed against actual project evidence before formal submission.